

# Laporan Tugas Kecil 3

IF2211 STRATEGI ALGORITMA  
Ice Sliding Puzzle Solver  
SEMESTER II TAHUN 2025/2026

Suryani Mulia Utami  
13524042

Syaqina Octavia Rizha  
13524088



LABORATORIUM ILMU DAN REKAVASA KOMPUTASI  
PROGRAM STUDI TEKNIK INFORMATIKA  
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA  
INSTITUT TEKNOLOGI BANDUNG

## DAFTAR ISI

|                                     |           |
|-------------------------------------|-----------|
| <b>BAB 1 PENDAHULUAN.....</b>       | <b>1</b>  |
| <b>BAB 2.....</b>                   | <b>2</b>  |
| <b>DASAR TEORI.....</b>             | <b>2</b>  |
| 2.1 Strategi Algoritma.....         | 2         |
| 2.2 Traversal Graf.....             | 2         |
| 2.3 Branch and Bound.....           | 2         |
| 2.4 Fungsi Heuristik.....           | 3         |
| <b>BAB 3.....</b>                   | <b>5</b>  |
| <b>DESAIN DAN IMPLEMENTASI.....</b> | <b>5</b>  |
| 3.3 Struktur Program.....           | 5         |
| 3.4 Cara Kerja Program.....         | 6         |
| 3.5 Algoritma Pathfinding.....      | 7         |
| 3.5.1 UCS.....                      | 9         |
| 3.5.2 GBFS.....                     | 9         |
| 3.5.3 A*.....                       | 10        |
| 3.6 Kompleksitas Algoritma.....     | 11        |
| 3.7 Kode Program.....               | 12        |
| BAB 4.....                          | 21        |
| EKSPERIMEN.....                     | 21        |
| <b>BAB 5.....</b>                   | <b>44</b> |
| <b>KESIMPULAN.....</b>              | <b>44</b> |
| <b>DAFTAR PUSTAKA.....</b>          | <b>45</b> |
| <b>LAMPIRAN.....</b>                | <b>46</b> |

## Pernyataan Kejujuran

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (*Generative AI*), melainkan hasil pemikiran dan analisis mandiri.



Suryani Mulia Utami



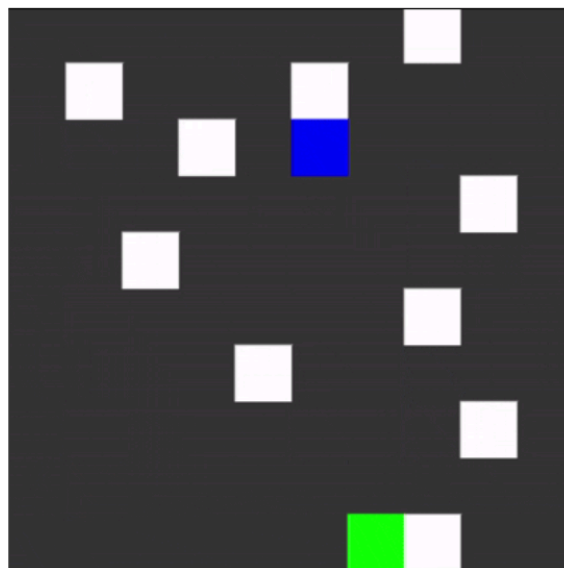
Syaqina Octavia Rizha

# BAB 1

## PENDAHULUAN

Permainan logika berbasis grid telah lama menjadi subjek kajian dalam ilmu komputer, khususnya dalam bidang kecerdasan buatan dan algoritma pencarian. Salah satu contoh permainan tersebut adalah Ice Sliding Puzzle, yaitu permainan logika di mana pemain harus menggerakkan karakter dari titik awal menuju titik keluar di atas permukaan es yang licin. Karakter hanya dapat bergerak secara horizontal atau vertikal, namun karena permukaan yang licin, karakter tidak akan berhenti bergerak hingga menabrak dinding atau rintangan. Mekanisme pergerakan yang deterministik namun tidak trivial ini menjadikan Ice Sliding Puzzle sebagai permasalahan pathfinding yang menarik untuk diselesaikan secara algoritmik.

Permasalahan pathfinding pada dasarnya adalah pencarian jalur terpendek atau teroptimal antara dua titik dalam suatu ruang pencarian. Dalam konteks Ice Sliding Puzzle, ruang pencarian tidak hanya ditentukan oleh posisi karakter pada papan, tetapi juga oleh urutan checkpoint yang harus dilalui secara berurutan sebelum mencapai titik tujuan. Hal ini memperluas ruang state yang harus dieksplorasi oleh algoritma, sehingga efisiensi pencarian menjadi faktor krusial.



Gambar 1: Visualisasi Permainan Ice Sliding Puzzle  
Sumber: Spesifikasi Tugas Kecil 3 IF2211 Semester II 2025/2026

## BAB 2

### DASAR TEORI

#### 2.1 Strategi Algoritma

Strategi algoritma merupakan suatu pendekatan umum yang digunakan dalam menyelesaikan permasalahan secara algoritmik, sehingga pendekatan tersebut dapat diterapkan pada berbagai persoalan komputasi. Beberapa strategi algoritma yang umum digunakan antara lain algoritma *brute force*, algoritma *greedy*, algoritma *divide and conquer*, algoritma *decrease and conquer*, algoritma *backtracking*, algoritma *branch and bound*, serta *dynamic programming*.

#### 2.2 Traversal Graf

Algoritma pencarian merupakan salah satu komponen penting dalam berbagai aplikasi komputasi. Berbagai algoritma telah dikembangkan untuk menangani pencarian pada beragam struktur data. Salah satu yang menjadi perhatian utama adalah pencarian pada graf karena luasnya penerapan, seperti pada struktur jaringan maupun representasi data hierarkis.

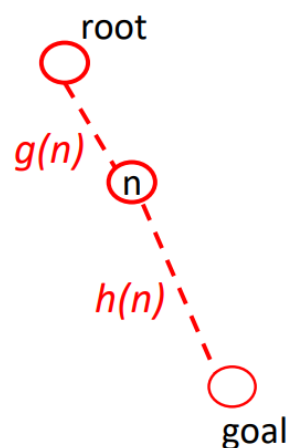
Traversal graf merupakan proses mengunjungi simpul-simpul di dalam graf secara sistematis. Mengunjungi simpul dapat berarti mencetak informasi yang terdapat pada simpul, memanipulasi nilainya, atau melakukan pengecekan tertentu. Dengan melakukan traversal, kita pada dasarnya melakukan pencarian solusi terhadap persoalan yang direpresentasikan dalam bentuk graf. Secara umum, algoritma pencarian berbasis graf dapat dibagi menjadi dua kategori, yaitu *uninformed search* (*blind search*) dan *informed search*. Contoh algoritma traversal *uninformed* adalah BFS, DFS, dan **Uniform Cost Search (UCS)**. Sedangkan contoh dari algoritma *informed* adalah **Greedy Best First Search (GBFS)** dan **A\***.

#### 2.3 Branch and Bound

Algoritma Branch and Bound merupakan strategi pencarian yang secara sistematis membagi ruang pencarian menjadi cabang-cabang (*branch*) sambil

memangkas cabang yang terbukti tidak dapat menghasilkan solusi lebih baik dari solusi terbaik yang sudah ditemukan (*bound*). UCS merupakan realisasi Branch and Bound pada graf berbobot, di mana eksplorasi diprioritaskan berdasarkan akumulasi biaya perjalanan dari titik awal. Efisiensi Branch and Bound dapat ditingkatkan lebih lanjut dengan memperketat nilai *bound* menggunakan informasi tambahan berupa fungsi heuristik yang mengestimasi biaya tersisa menuju tujuan. Pendekatan inilah yang melahirkan algoritma informed search seperti GBFS dan A\*.

## 2.4 Fungsi Heuristik



Gambar 2: Visualisasi komponen fungsi evaluasi/*bound*

Sumber: Rinaldi Munir. Route Planning (Bagian 2)

*Heuristic function* ( $h(n)$ ) adalah informasi tambahan yang digunakan untuk memperkirakan seberapa dekat suatu simpul ke tujuan, atau dengan kata lain simpul mana yang “lebih menjanjikan” untuk dieksplorasi terlebih dahulu. Dalam kerangka Branch and Bound,  $h(n)$  berperan memperketat nilai *bound*. Alih-alih hanya menggunakan biaya aktual  $g(n)$  yang sudah ditempuh, *bound* kini diperkaya menjadi  $g(n) + h(n)$  sehingga pemangkasan cabang dapat dilakukan lebih agresif dan pencarian menjadi lebih efisien.

Agar pemangkasan tersebut tidak menghilangkan solusi optimal, fungsi heuristik yang digunakan haruslah *admissible*, yaitu tidak pernah melebihi biaya

aktual menuju tujuan. Secara formal,  $h(n)$  *admissible* jika untuk setiap simpul  $n$  berlaku  $h(n) \leq h^*(n)$ , di mana  $h^*(n)$  adalah biaya aktual dari simpul  $n$  menuju tujuan.

if  $f(n) = g(n) \rightarrow$  Uniform Cost Search (UCS)

if  $f(n) = h(n) \rightarrow$  Greedy Best First Search

if  $f(n) = g(n) + h(n) \rightarrow A^*$

Gambar 3: Perbedaan UCS, GBFS, dan A\*  
Sumber: Rinaldi Munir. Route Planning (Bagian 2)

## BAB 3

### DESAIN DAN IMPLEMENTASI

#### 3.3 Struktur Program

Berikut adalah struktur dari Program Ice Sliding Puzzle Solver:

```
Tucil3_13524042_13524088/
├── CMakeLists.txt
├── README.md
├── assets/
├── src/
│   ├── main.cpp
│   ├── graph.cpp
│   ├── gui.cpp
│   ├── heuristikCost.cpp
│   ├── loadSave.cpp
│   ├── searchAlgorithm.cpp
│   └── include/
│       ├── board.hpp
│       ├── graph.hpp
│       ├── gui.hpp
│       ├── heuristikCost.hpp
│       ├── loadSave.hpp
│       ├── searchAlgorithm.hpp
│       └── state.hpp
└── test/
    └── input/
        ├── test1.txt
        ├── test2.txt
        ├── test3.txt
        ├── test4.txt
        ├── test5.txt
        ├── test6.txt
        ├── test14.txt
        ├── test25.txt
        ├── test50_1.txt
        └── test50_2.txt
```



### 3.4 Cara Kerja Program

Berikut adalah tahapan cara kerja program, mulai dari membaca file hingga menuliskan file hasil pencarian.

Table 1: Tahapan Cara Kerja Program

| No | Nama Fungsi/Method                                                | Deskripsi                                                                                                                      | Output                                                                                                                             |
|----|-------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| 1  | void loadFile(string filename, Board* mainBoard)                  | Membaca file .txt dan mengisi struct Board dengan konfigurasi papan permainan. Dan melakukan validasi ukuran papan             | Struct Board terisi dengan grid, cost, start, goal, dan checkpoints[0..9]                                                          |
| 2  | optional<Edge> getSlideEdge(const Board& board, Node n, char dir) | Mensimulasikan satu gerakan sliding dari node n ke arah dir, dan menghasilkan edge jika gerakan tersebut valid.                | Berisi edge jika valid, nullopt jika path tidak valid (keluar map atau melewati lava). Menjadi fungsi helper untuk membangun graf. |
| 3  | void buildGraph(const Board& board, Graph& graph)                 | Mengonversi representasi Board menjadi struktur graf berarah berbobot yang merepresentasikan semua gerakan sliding yang valid. | Struct Graph berisi nodes, adj, rev_adj, start, goal, num_checkpoints                                                              |
| 4  | SearchResult searchUCS(const Board& board, const Graph& graph)    | Mencari jalur solusi dari start ke goal menggunakan Uniform Cost Search.                                                       | SearchResult berisi found, totalCost, iterations, path (urutan edge solusi).                                                       |
| 5  | SearchResult searchGBFS(const Board& board, const Graph& graph)   | Mencari jalur solusi dari start ke goal menggunakan Greedy Best First Search                                                   | SearchResult berisi found, totalCost, iterations, path (urutan edge solusi) terisi.                                                |
| 6  | SearchResult searchAStar(const Board& board, const Graph& graph)  | Mencari jalur solusi dari start ke goal menggunakan A*                                                                         | SearchResult berisi found, totalCost, iterations, path (urutan edge solusi).                                                       |
| 7  | optional<State> nextState(const Board& board, const State& curr,  | Menghitung state berikutnya setelah satu gerakan sliding. Digunakan untuk memvalidasi urutan checkpoint.                       | Berisi state baru jika valid, nullopt jika melanggar urutan checkpoint                                                             |

|   |                                            |                                            |                                                                  |
|---|--------------------------------------------|--------------------------------------------|------------------------------------------------------------------|
|   | const Edge& edge)                          |                                            |                                                                  |
| 8 | void<br>saveResult(const<br>GuiState& gui) | Menyimpan hasil pencarian<br>ke file .txt. | File .txt tersimpan di<br>direktori output yang<br>dipilih user. |

### 3.5 Algoritma Pathfinding

Algoritma pathfinding yang diimplementasikan menggunakan strategi pencarian branch and bound, di mana ruang pencarian direpresentasikan sebagai graf berarah berbobot dengan node merepresentasikan state permainan dan edge merepresentasikan gerakan sliding yang valid. Pada setiap langkah, algoritma meng-*expand* node yang memiliki nilai fungsi evaluasi  $f(n)$  minimum dari antrian prioritas, kemudian memangkas cabang yang terbukti tidak dapat menghasilkan solusi lebih baik dari solusi terbaik yang sudah ditemukan.

Dalam implementasinya, ketiga algoritma pathfinding diimplementasikan menggunakan priority queue yang menyimpan struktur data `NodeStateQueue` sebagai berikut untuk melacak posisi aktor dan urutan checkpoint yang sudah dilewati:

```
struct State{

    Node pos;

    int nextCheckpoint;

}

struct NodeStateQueue{

    State state;

    float gCost;

    float hCost;

    float fCost;
```

```

bool operator>(const NodeStateQueue& other) const{

    if (fCost != other.fCost) {return fCost > other.fCost;}

    //kalau tie

    return gCost < other.gCost;

}

};

```

Secara umum, ketiga algoritma mengikuti langkah-langkah berikut:

1. **Inisiasi.** Bentuk state awal dari posisi start dengan nextCheckpoint = 0. Hitung nilai  $f(\text{start})$  sesuai fungsi evaluasi algoritma yang digunakan, kemudian push ke min-priority queue.
2. **Iterasi.** Selama priority queue tidak kosong, pop state dengan  $f(n)$  terkecil. Jika state yang di-pop merupakan stale entry, yaitu sudah ada path lebih murah ke state ini yang ditemukan setelahnya, maka langsung dilewati tanpa di-expand.
3. **Pengecekan tujuan.** Jika state yang di-expand adalah goal state (posisi aktor di tile goal dan nextCheckpoint  $\geq$  num\_checkpoints), solusi ditemukan. Path direkonstruksi secara mundur dari goal ke start menggunakan tabel parentState dan parentEdge, kemudian dibalik urutannya.
4. **Ekspansi.** Jika bukan goal state, ekspansi semua edge dari posisi saat ini. Untuk setiap edge, nextState() dipanggil untuk menghitung state berikutnya sekaligus memvalidasi urutan checkpoint. Edge yang melanggar urutan checkpoint langsung dilewati. Neighbor yang valid dievaluasi: jika memenuhi kriteria penerimaan algoritma, push ke priority queue dan perbarui tabel parent.
5. **Terminasi.** Algoritma berhenti saat goal ditemukan atau priority queue kosong. Jika queue habis tanpa menemukan goal, solusi tidak ditemukan.

Perbedaan antara ketiga algoritma terletak pada fungsi evaluasi  $f(n)$  dan kriteria penerimaan neighbor.

### 3.5.1 UCS

UCS adalah algoritma *pathfinding* uninformed yang menggunakan fungsi evaluasi  $f(n) = g(n)$ , yaitu hanya mempertimbangkan biaya aktual dari root ke node  $n$  tanpa estimasi heuristik. Karena tidak memanfaatkan informasi arah menuju tujuan, UCS mengeksplorasi ruang pencarian secara merata ke segala arah berdasarkan akumulasi biaya perjalanan, sehingga menjamin ditemukannya solusi optimal pada graf berbobot positif.

Pada implementasinya, UCS menggunakan *priority queue* untuk menyimpan simpul. Simpul dengan biaya kumulatif terendah diperluas terlebih dahulu. UCS menghitung biaya kumulatif dari simpul awal ke simpul saat ini dan memprioritaskan simpul dengan biaya yang lebih rendah. Neighbor hanya akan dipush ke dalam queue apabila cost  $g(\text{neighbor})$  lebih kecil dari biaya yang pernah tercatat sebagai best distance sebelumnya. Algoritma berhenti ketika simpul tujuan diperluas, memastikan bahwa saat pertama kali simpul tujuan tercapai, jalur tersebut adalah jalur yang optimal.

Pada algoritma ini, didefinisikan bahwa  $f(n) = g(n)$ , dimana  $g(n)$  adalah biaya dari *root* hingga ke  $n$  dan  $f(n)$  adalah nilai prioritas node pada *priority queue*.

### 3.5.2 GBFS

GBFS adalah algoritma *pathfinding* informed yang menggunakan fungsi evaluasi  $f(n) = h(n)$ , yaitu hanya mempertimbangkan estimasi heuristik biaya tersisa dari node  $n$  menuju goal tanpa memperhitungkan biaya aktual yang sudah ditempuh. Algoritma ini bersifat *greedy* karena selalu memprioritaskan node yang diperkirakan paling dekat ke tujuan, sehingga cenderung menemukan solusi lebih cepat dibandingkan UCS, namun tidak menjamin solusi optimal karena biaya aktual diabaikan dalam prioritas eksplorasi.

Algoritma ini tidak melakukan *backtracking* atau mempertimbangkan kembali node yang tampak kurang optimal, sehingga cepat tetapi berpotensi tidak lengkap jika jalur yang dipilih tidak mengarah ke solusi. Pada implementasinya, neighbor hanya akan dipush ke dalam queue apabila belum pernah di-*discover* (tidak ada re-visit). Algoritma ini sepenuhnya dipandu heuristik tanpa mempertimbangkan biaya aktual. Fungsi evaluasi  $f(n) = h(n)$ , dengan  $h(n)$  adalah nilai perkiraan biaya dari  $n$  ke tujuan yang ditentukan berdasarkan fungsi heuristik. Pada program ini, heuristik yang digunakan sama seperti A\* yaitu *Manhattan*, *Euclidean* dan *Minimum Slide Cost*.

### 3.5.3 A\*

A\* adalah algoritma pathfinding *informed* yang menggunakan fungsi evaluasi  $f(n) = g(n) + h(n)$ . Algoritma ini menggabungkan biaya aktual atau biaya dari *root*  $g(n)$ , dan estimasi heuristik atau estimasi biaya dari posisi saat ini ke node goal  $h(n)$  dengan mempertimbangkan aturan checkpoint. Hal ini memungkinkan pemangkasan cabang dapat dilakukan lebih agresif. Dalam implementasinya, neighbor hanya akan dipush ke dalam queue apabila cost  $g(\text{neighbor})$  lebih kecil dari biaya yang pernah tercatat sebagai best distance sebelumnya.

Algoritma A\* menjamin ditemukannya solusi optimal dengan syarat nilai  $f(n)$  harus merupakan *lower bound* yang valid dari biaya solusi sesungguhnya yang melewati node  $n$ . Syarat ini terpenuhi selama fungsi heuristik  $h(n)$  bersifat *admissible*. Fungsi heuristik yang diimplementasikan di antaranya Manhattan distance, Euclidean distance, serta Minimum Slide Cost, merupakan fungsi heuristik yang *admissible*, karena costnya dipastikan kurang dari actual cost. Ketiga heuristik ini mengestimasi biaya perjalanan secara geometris tanpa mempertimbangkan mekanisme sliding maupun variasi biaya tile pada papan.

Pada Manhattan distance, diasumsikan pemain dapat berpindah posisi secara bebas ke arah horizontal dan vertikal dengan biaya seragam per langkah, tanpa mempertimbangkan mekanisme sliding maupun biaya tile

yang sebenarnya. Estimasi jarak dihitung  $h(n) = |\Delta_{row}| + |\Delta_{col}|$ . Karena mekanisme sliding pada kenyataannya mengharuskan pemain melewati beberapa tile sekaligus dengan biaya yang bervariasi, biaya aktual selalu lebih besar atau sama dengan estimasi Manhattan, sehingga *admissible*.

Pada Euclidean distance, jarak diestimasi sebagai garis lurus antara posisi saat ini dengan target, yaitu  $h(n) = \sqrt{(\Delta_{row}^2 + \Delta_{col}^2)}$ . Euclidean distance selalu lebih kecil atau sama dengan Manhattan distance untuk grid movement, sehingga heuristik ini lebih lemah dari Manhattan dalam hal informativeness, namun tetap *admissible*.

Minimum Slide Cost merupakan pengembangan dari Manhattan distance yang mempertimbangkan variasi biaya tile pada papan. Dengan setiap perpindahan menggunakan biaya tile minimum di baris A untuk komponen horizontal dan biaya tile minimum di kolom A untuk komponen vertikal, sehingga estimasi yang dihasilkan selalu lebih besar atau sama dengan Manhattan distance murni namun tetap *admissible* karena menggunakan biaya yang lebih kecil dari aktual tile yang dilalui selama sliding.

### 3.6 Kompleksitas Algoritma

Berikut ini akan dijelaskan terkait *time complexity* dari tiap algoritma. Kemudian ini adalah variable yang digunakan:

E = jumlah edge total di graph (maksimal 4: L/R/U/D)

V = jumlah state unik yang mungkin = jumlah node di graph x (jumlah checkpoint + 1)

d = jumlah step pada solusi optimal

#### 1. UCS

Setiap state bisa masuk priority queue berkali-kali (tiap kali ada path baru ke sana). Namun tiap state hanya di-expand sekali (saat di-pop dengan cost terkecil).

$$T_{UCS} = O(E \times \log V)$$

## 2. GBFS

GBFS tidak melakukan re-visit. Setiap state di-discover sekali saja. Dengan tidak re-open, GBFS jauh lebih cepat tapi bisa menemukan path yang bukan paling optimal.

$$T_{GBFS} = O(E \times \log V)$$

Tapi dalam aplikasinya algoritma ini bisa lebih cepat karena tidak re-visit sehingga *priority queue* bisa jauh lebih kecil. Kemudian jika heuristiknya bagus maka bisa lebih cepat mencapai node tujuan

## 3. A\*

Algoritma ini memiliki kemiripan dengan UCS. A\* perlu re-visit. Kalau ada path lebih murah ke state yang sudah di-discover, state itu akan dikunjungi kembali.

Best case (heuristik sempurna  $h = h^*$ ), iterasi akan langsung sesuai melangkah sesuai solusi optimal:

$$T_{A^*} = O(d)$$

Average case dengan admissible  $h$  (hasil akan optimal):

$$T_{A^*} = O(E \times \log V)$$

## 3.7 Kode Program

### Algoritma Pathfinding UCS

Nama File: src/searchAlgorithm.cpp

```
SearchResult SearchAlgorithm::searchUCS(const Board& board, const Graph& g){
    unordered_map<State, State, StateHash> parentState; //untuk
reconstruct path
    unordered_map<State, Edge, StateHash> parentEdge; //untuk
reconstruct path
    // Catat informasi search
    SearchResult res;
    res.found = false;
    res.totalCost = 0;
```

```

res.iterations = 0;
res.startPos = g.start;

// Catat visited Node beserta costnya
unordered_map<State, float, StateHash> visitedNode;

// Min priority queue
std::priority_queue<NodeStateQueue, vector<NodeStateQueue>,
greater<NodeStateQueue>> priorQ;

// Push start node ke queue
State startState;
startState.pos = g.start;
startState.nextCheckpoint = 0;
float gn = 0.0;
float fn = gn;

priorQ.push({startState, gn, 0, fn}); //hn = 0

visitedNode[startState] = 0;

// Proses sampai kosong atau sampai ketemu goal
while (!(priorQ.empty())){
    NodeStateQueue currNQ = priorQ.top(); //expand node yang punya
cost minimum
    priorQ.pop();
    State currState = currNQ.state;
    float currCost = currNQ.gCost; //cumulative cost sampai
currState

    if (visitedNode.count(currState) && currCost >
visitedNode[currState]) continue; //udah pernah dilewatn dengan cost
lebih baik

    res.iterations++;

    // Kalau udah ketemu goal, berhenti
    if (currState.pos == g.goal && allCheckpointVisited(currState,
g)){
        res.found = true;
        res.totalCost = currCost;
        res.path = reconstructPath(parentState, parentEdge,

```



```

startState, currState);

    return res;
}
// Daftarkan node tetangga ke queue
for (const Edge& e : g.adj.at(currState.pos)){
    auto maybeN = nextState(board, currState, e);
    if (!maybeN) continue; //slide melanggar urutan checkpoint =
game over, skip
    State nState = *maybeN;
    gn = currCost + e.cost; //parent cost + edge cost
    fn = gn;
    if (!visitedNode.count(nState) || fn < visitedNode[nState]){
//blm pernah dilewatin atau costnya lebih baik
        visitedNode[nState] = fn;
        parentState[nState] = currState;
        parentEdge[nState] = e;

        priorQ.push({nState, gn, 0, fn});
    }
}
// priorQ.empty(), not found
return res;
}

```

### Algoritma Pathfinding GBFS

Nama File: src/searchAlgorithm.cpp

```

SearchResult SearchAlgorithm::searchGBFS(const Board& board, const Graph&
g){
    unordered_map<State, State, StateHash> parentState;
    unordered_map<State, Edge, StateHash> parentEdge;
    // Catat informasi search
    SearchResult res;
    res.found = false;
    res.totalCost = 0;
    res.iterations = 0;
    res.startPos = g.start;
}

```

```

// GBFS: tiap state cukup di-discover sekali (gak ada re-open)
unordered_set<State, StateHash> discovered;

// Min priority queue berdasarkan h(n)
std::priority_queue<NodeStateQueue, vector<NodeStateQueue>,
greater<NodeStateQueue>> priorQ;

// Push start node ke queue
State startState;
startState.pos = g.start;
startState.nextCheckpoint = 0;

float h0 = hCost->cost(board, g, startState);
discovered.insert(startState);
priorQ.push({startState, 0.0f, h0, h0}); //fCost = h(n)

// Proses sampai kosong atau sampai ketemu goal
while (!(priorQ.empty())){
    NodeStateQueue currNQ = priorQ.top(); //expand node yang punya h
    paling kecil
    priorQ.pop();
    State currState = currNQ.state;
    float currG = currNQ.gCost; //cumulative cost sampai currState
    (untuk reporting)

    res.iterations++;

    // Kalau udah ketemu goal, berhenti
    if (currState.pos == g.goal && allCheckpointVisited(currState,
g)){
        res.found = true;
        res.totalCost = currG;
        res.path = reconstructPath(parentState, parentEdge,
startState, currState);

        return res;
    }
    // Daftarkan node tetangga ke queue
    for (const Edge& e : g.adj.at(currState.pos)){
        auto maybeN = nextState(board, currState, e);
        if (!maybeN) continue; //slide melanggar urutan checkpoint =

```

```

game over, skip
    State nState = *maybeN;
    if (discovered.count(nState)) continue; //GBFS: skip kalau
    udah pernah di-discover

    discovered.insert(nState);
    float gn = currG + e.cost;
    float hn = hCost->cost(board, g, nState);

    parentState[nState] = currState;
    parentEdge[nState] = e;
    priorQ.push({nState, gn, hn, hn}); //fCost = h(n)
}
}
// priorQ.empty(), not found
return res;
}

```

### Algoritma Pathfinding A\*

**Nama File:** src/searchAlgorithm.cpp

```

SearchResult SearchAlgorithm::searchAStar(const Board& board, const
Graph& g){
    unordered_map<State, State, StateHash> parentState;
    unordered_map<State, Edge, StateHash> parentEdge;
    // Catat informasi search
    SearchResult res;
    res.found = false;
    res.totalCost = 0;
    res.iterations = 0;
    res.startPos = g.start;

    // Catat best gCost yang pernah didapat per state (untuk re-open)
    unordered_map<State, float, StateHash> bestG;

    // Min priority queue berdasarkan  $f(n) = g(n) + h(n)$ 
    std::priority_queue<NodeStateQueue, vector<NodeStateQueue>,
greater<NodeStateQueue>> priorQ;

    // Push start node ke queue

```

```

    State startState;
    startState.pos = g.start;
    startState.nextCheckpoint = 0;

    float h0 = hCost->cost(board, g, startState);
    bestG[startState] = 0.0f;
    priorQ.push({startState, 0.0f, h0, h0}); //fCost = 0 + h0

    // Proses sampai kosong atau sampai ketemu goal
    while (!(priorQ.empty())){
        NodeStateQueue currNQ = priorQ.top(); //expand node yang punya f
minimum
        priorQ.pop();
        State currState = currNQ.state;
        float currG = currNQ.gCost;

        if (currG > bestG[currState]) continue; //stale entry, udah ada
path lebih murah ke state ini

        res.iterations++;

        // Kalau udah ketemu goal, berhenti
        if (currState.pos == g.goal && allCheckpointVisited(currState,
g)){
            res.found = true;
            res.totalCost = currG;
            res.path = reconstructPath(parentState, parentEdge,
startState, currState);

            return res;
        }
        // Daftarkan node tetangga ke queue
        for (const Edge& e : g.adj.at(currState.pos)){
            auto maybeN = nextState(board, currState, e);
            if (!maybeN) continue; //slide melanggar urutan checkpoint =
game over, skip
            State nState = *maybeN;
            float gn = currG + e.cost;

            if (!bestG.count(nState) || gn < bestG[nState]){ //blm
pernah dilewatkan atau costnya lebih baik
                bestG[nState] = gn;
            }
        }
    }

```

```

        float hn = hCost->cost(board, g, nState);
        float fn = gn + hn;

        parentState[nState] = currState;
        parentEdge[nState] = e;
        priorQ.push({nState, gn, hn, fn});
    }
}
// priorQ.empty(), not found
return res;
}

```

### Algoritma Pathfinding GBFS

**Nama File:** src/searchAlgorithm.cpp

```

SearchResult SearchAlgorithm::searchGBFS(const Board& board, const Graph&
g){
    unordered_map<State, State, StateHash> parentState;
    unordered_map<State, Edge, StateHash> parentEdge;
    // Catat informasi search
    SearchResult res;
    res.found = false;
    res.totalCost = 0;
    res.iterations = 0;
    res.startPos = g.start;

    // GBFS: tiap state cukup di-discover sekali (gak ada re-open)
    unordered_set<State, StateHash> discovered;

    // Min priority queue berdasarkan h(n)
    std::priority_queue<NodeStateQueue, vector<NodeStateQueue>,
greater<NodeStateQueue>> priorQ;

    // Push start node ke queue
    State startState;
    startState.pos = g.start;
    startState.nextCheckpoint = 0;

```

```

float h0 = hCost->cost(board, g, startState);
discovered.insert(startState);
priorQ.push({startState, 0.0f, h0, h0}); //fCost = h(n)

// Proses sampai kosong atau sampai ketemu goal
while (!(priorQ.empty())){
    NodeStateQueue currNQ = priorQ.top(); //expand node yang punya h
    paling kecil
    priorQ.pop();
    State currState = currNQ.state;
    float currG = currNQ.gCost; //cumulative cost sampai currState
    (untuk reporting)

    res.iterations++;

    // Kalau udah ketemu goal, berhenti
    if (currState.pos == g.goal && allCheckpointVisited(currState,
g)){
        res.found = true;
        res.totalCost = currG;
        res.path = reconstructPath(parentState, parentEdge,
startState, currState);

        return res;
    }
    // Daftarkan node tetangga ke queue
    for (const Edge& e : g.adj.at(currState.pos)){
        auto maybeN = nextState(board, currState, e);
        if (!maybeN) continue; //slide melanggar urutan checkpoint =
game over, skip
        State nState = *maybeN;
        if (discovered.count(nState)) continue; //GBFS: skip kalau
udah pernah di-discover

        discovered.insert(nState);
        float gn = currG + e.cost;
        float hn = hCost->cost(board, g, nState);

        parentState[nState] = currState;
        parentEdge[nState] = e;
        priorQ.push({nState, gn, hn, hn}); //fCost = h(n)
    }
}

```

```
}  
// priorQ.empty(), not found  
return res;  
}
```

## BAB 4

### EKSPERIMEN

Table 2: Hasil Eksekusi Program

❖ Kasus Uji 1

Input

7 7  
XXXXXXX  
X0\*\*\*\*X  
X\*\*X\*\*X  
X\*\*\*\*OX  
X1\*\*\*LX  
XZ\*\*X\*X  
XXXXXXX  
999 999 999 999 999 999 999  
999 3 5 2 8 1 999  
999 7 4 999 6 9 999  
999 2 8 3 5 4 999  
999 6 1 7 2 999 999  
999 9 3 4 999 8 999  
999 999 999 999 999 999 999

Output UCS

Slider Puzzle Solver

ICE SLIDING  
PUZZLE SOLVER

INPUT FILE  
test1.txt

ALGORITHM  
UCS  
GBFS  
A\*

HEURISTIC  
H1 - Manhattan  
H2 - Euclidean  
H3 - Minslidecost

RUN

SAVE RESULT

SOLUSI DITEMUKAN!

|   | 0 | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|---|
| 0 |   |   |   |   |   |   |   |
| 1 |   | 0 |   |   |   |   |   |
| 2 |   |   |   |   |   |   |   |
| 3 |   |   |   |   |   |   |   |
| 4 |   | 1 |   |   |   |   |   |
| 5 |   | 2 |   |   |   |   |   |
| 6 |   |   |   |   |   |   |   |

STEP 0 / 8

<

>

>>

SPEED: 2X 4X 8X 32X

RESULTS

CONFIGURATION  
Algorithm : UCS

METRICS  
Status : DITEMUKAN  
Waktu : 0.21 ms  
Iterasi : 13  
Total Cost: 87  
Langkah : 8

URUTAN GERAKAN  
RULUDRUR

DETAIL LANGKAH  
STEP : 1 / 8  
Arah : KANAN (R)  
DARI : (5, 1)  
ke : (5, 3)  
Cost : 7  
Total Cost: 7



## Output GBFS

**Slider Puzzle Solver**

**ICE SLIDING PUZZLE SOLVER**

**INPUT FILE**  
test1.txt

**ALGORITHM**  
☐ UCS  
☒ GBFS  
☐ A\*

**HEURISTIC**  
☒ H1 - Mahhattan  
☐ H2 - Euclidean  
☐ H3 - Minslidecost

**RUN**  
**SAVE RESULT**

**SOLUSI ditemukan!**

**RESULTS**

**CONFIGURATION**  
Algorithm: GBFS + H1 (Mahhattan)

**METRICS**  
 Status : DITEMUKAN  
 Waktu : 0.19 ms  
 Iterasi : 9  
 Total Cost: 87  
 Langkah : 8

**URUTAN GERAKAN**  
RULUDRUR

**DETAIL LANGKAH**  
 Step : 1 / 8  
 Arah : KANAN (R)  
 Dari : (5, 1)  
 Ke : (5, 3)  
 Cost : 7  
 Total Cost: 7

Step 0 / 8

Speed: 2X 4X 8X 32X

## Output A\*

**Slider Puzzle Solver**

**ICE SLIDING PUZZLE SOLVER**

**INPUT FILE**  
test1.txt

**ALGORITHM**  
☐ UCS  
☒ GBFS  
☐ A\*

**HEURISTIC**  
☒ H1 - Mahhattan  
☐ H2 - Euclidean  
☐ H3 - Minslidecost

**RUN**  
**SAVE RESULT**

**SOLUSI ditemukan!**

**RESULTS**

**CONFIGURATION**  
Algorithm: GBFS + H1 (Mahhattan)

**METRICS**  
 Status : DITEMUKAN  
 Waktu : 0.19 ms  
 Iterasi : 9  
 Total Cost: 87  
 Langkah : 8

**URUTAN GERAKAN**  
RULUDRUR

**DETAIL LANGKAH**  
 Step : 1 / 8  
 Arah : KANAN (R)  
 Dari : (5, 1)  
 Ke : (5, 3)  
 Cost : 7  
 Total Cost: 7

Step 0 / 8

Speed: 2X 4X 8X 32X

## ❖ Kasus Uji 2

| Input |
|-------|
| 8 8   |

```

XXXXXXXXX
X***X**X
X**X***X
X*L***X
XX****LX
X*O*ZX*X
XXXXXXXXX
XXXXXXXXX

999 999 999 999 999 999 999 999
  999 8 8 8 999 8 8 999
    999 8 8 999 8 8 8 999
      999 8 7 8 8 8 8 999
        999 999 8 8 8 8 7 999
          999 8 8 8 8 999 8 999
            999 999 999 999 999 999 999
              999 999 999 999 999 999 999

```

## Output UCS

Slider Puzzle Solver

### ICE SLIDING PUZZLE SOLVER

INPUT FILE

test2.txt

ALGORITHM

☒ UCS
 ☐ GBFS
 ☐ A\*

HEURISTIC

☐ H1 - Manhattan
 ☐ H2 - Euclidean
 ☐ H3 - Minslidelcost

RUN

SAVE RESULT

Solusi ditemukan!

|   | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|---|
| 0 |   |   |   |   |   |   |   |   |
| 1 |   |   |   |   |   |   |   |   |
| 2 |   |   |   |   |   |   |   |   |
| 3 |   |   | L |   |   |   |   |   |
| 4 |   |   |   |   |   |   |   |   |
| 5 |   |   | O |   |   |   |   |   |
| 6 |   |   |   |   |   |   |   |   |
| 7 |   |   |   |   |   |   |   |   |

STEP 0 / 7

< > >>

SPEED: 25% 50% 75% 100%

### RESULTS

CONFIGURATION

Algorithm : UCS

METRICS

Status : DITEMUKAN  
 Waktu : 0.18 ms  
 Iterasi : 9  
 Total Cost: 112  
 Langkah : 7

URUTAN GERAKAN

URULDL

DETAIL LANGKAH

Step : 1 / 7  
 Arah : ATAS (U)  
 Dari : (5, 4)  
 Ke : (2, 4)  
 Cost : 24  
 Total Cost: 24

## Output GBFS

Slider Puzzle Solver

ICE SLIDING PUZZLE SOLVER

INPUT FILE

test2.txt

ALGORITHM

☐ UCS
☒ GBFS
☐ A\*

HEURISTIC

☐ H1 - Manhattan
☒ H2 - Euclidean
☐ H3 - Minslidelcost

RUN

SAVE RESULT

solusi ditemukan!

|   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|
| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
| 0 |   |   |   |   |   |   |   |
| 1 |   |   |   |   |   |   |   |
| 2 |   |   |   |   |   |   |   |
| 3 |   |   | L |   |   |   |   |
| 4 |   |   |   |   |   |   |   |
| 5 |   |   | O |   |   |   |   |
| 6 |   |   |   |   |   |   |   |
| 7 |   |   |   |   |   |   |   |

STEP 0 / 7

<

>

>>

SPEED: 2X 4X 8X 32X

RESULTS

CONFIGURATION

Algorithm : GBFS + H2 (Euclidean)

METRICS

Status : DITEMUKAN

Waktu : 0.20 ms

Iterasi : 9

Total Cost: 112

Langkah : 7

URUTAN GERAKAN

URULDL

DETAIL LANGKAH

Step : 1 / 7

Arah : ATAS (U)

Dari : (5, 4)

Ke : (2, 4)

Cost : 24

Total Cost: 24

Output A\*

Slider Puzzle Solver

ICE SLIDING PUZZLE SOLVER

INPUT FILE

test2.txt

ALGORITHM

☐ UCS
☐ GBFS
☒ A\*

HEURISTIC

☐ H1 - Manhattan
☐ H2 - Euclidean
☒ H3 - Minslidelcost

RUN

SAVE RESULT

solusi ditemukan!

|   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|
| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
| 0 |   |   |   |   |   |   |   |
| 1 |   |   |   |   |   |   |   |
| 2 |   |   |   |   |   |   |   |
| 3 |   |   | L |   |   |   |   |
| 4 |   |   |   |   |   |   |   |
| 5 |   |   | O |   |   |   |   |
| 6 |   |   |   |   |   |   |   |
| 7 |   |   |   |   |   |   |   |

STEP 0 / 7

<

>

>>

SPEED: 2X 4X 8X 32X

RESULTS

CONFIGURATION

Algorithm : A\* + H3 (Minslidelcost)

METRICS

Status : DITEMUKAN

Waktu : 0.15 ms

Iterasi : 9

Total Cost: 112

Langkah : 7

URUTAN GERAKAN

URULDL

DETAIL LANGKAH

Step : 1 / 7

Arah : ATAS (U)

Dari : (5, 4)

Ke : (2, 4)

Cost : 24

Total Cost: 24

### ❖ Kasus Uji 3

|                                     |
|-------------------------------------|
| Input                               |
| <div>8 8</div> <div>XXXXXXXXX</div> |

```

X***X2*X
X**X0*1X
X*L****X
XX4**3LX
X*O*ZX*X
XXXXXXXXX
XXXXXXXXX
999 999 999 999 999 999 999 999
999 8 8 8 999 8 8 999
999 8 8 999 8 8 8 999
999 8 7 8 8 8 8 999
999 999 8 8 8 8 7 999
999 8 8 8 8 999 8 999
999 999 999 999 999 999 999 999
999 999 999 999 999 999 999 999

```

## Output UCS

Slider Puzzle Solver

### ICE SLIDING PUZZLE SOLVER

INPUT FILE

test3.txt

ALGORITHM

☒ UCS

☐ GBFS

☐ A\*

HEURISTIC

☐ H1 - Manhattan

☐ H2 - Euclidean

☐ H3 - Minslidecost

RUN

SAVE RESULT

Solusi ditemukan!

|   | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|---|
| 0 |   |   |   |   |   |   |   |   |
| 1 |   |   |   |   |   | 2 |   |   |
| 2 |   |   |   |   | 0 |   | 1 |   |
| 3 |   |   | L |   |   |   |   |   |
| 4 |   |   | 4 |   |   | 3 | L |   |
| 5 |   |   | 0 |   | 2 |   |   |   |
| 6 |   |   |   |   |   |   |   |   |
| 7 |   |   |   |   |   |   |   |   |

Step 0 / 7

Speed: 2X 4X 8X 32X

### RESULTS

CONFIGURATION

Algorithm: UCS

METRICS

Status: DITEMUKAN

Waktu: 0.18 ms

Iterasi: 17

Total Cost: 112

Langkah: 7

URUTAN GERAKAN

URULDL

DETAIL LANGKAH

Step: 1 / 7

Arah: ATAS (U)

Dari: (5, 4)

Ke: (2, 4)

Cost: 24

Total Cost: 24

## Output GBFS

Slider Puzzle Solver

ICE SLIDING  
PUZZLE SOLVER

INPUT FILE  
test3.txt

ALGORITHM  
☐ UCS  
☒ GBFS  
☐ A\*

HEURISTIC  
☒ H1 - Manhattan  
☐ H2 - Euclidean  
☐ H3 - Minslidecost

RUN

SAVE RESULT

Solusi ditemukan!

|   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|
| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
| 0 |   |   |   |   |   |   |   |
| 1 |   |   |   |   | 2 |   |   |
| 2 |   |   |   | 0 |   | 1 |   |
| 3 |   | L |   |   |   |   |   |
| 4 |   | 4 |   |   | 3 | L |   |
| 5 |   | 0 |   | 2 |   |   |   |
| 6 |   |   |   |   |   |   |   |
| 7 |   |   |   |   |   |   |   |

STEP 0 / 7

<

>

>>

SPEED: 2X 4X 8X 32X

RESULTS

CONFIGURATION  
Algorithm : GBFS + H1 (Manhattan)

METRICS  
 Status : DITEMUKAN  
 Waktu : 0.19 ms  
 Iterasi : 8  
 Total Cost: 112  
 Langkah : 7

URUTAN GERAKAN  
URULDL

DETAIL LANGKAH  
 Step : 1 / 7  
 arah : ATAS (U)  
 dari : (5, 4)  
 ke : (2, 4)  
 cost : 24  
 Total Cost: 24

Output A\*

Slider Puzzle Solver

ICE SLIDING  
PUZZLE SOLVER

INPUT FILE  
test3.txt

ALGORITHM  
☐ UCS  
☒ GBFS  
☐ A\*

HEURISTIC  
☐ H1 - Manhattan  
☒ H2 - Euclidean  
☐ H3 - Minslidecost

RUN

SAVE RESULT

Solusi ditemukan!

|   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|
| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
| 0 |   |   |   |   |   |   |   |
| 1 |   |   |   |   | 2 |   |   |
| 2 |   |   |   | 0 |   | 1 |   |
| 3 |   | L |   |   |   |   |   |
| 4 |   | 4 |   |   | 3 | L |   |
| 5 |   | 0 |   | 2 |   |   |   |
| 6 |   |   |   |   |   |   |   |
| 7 |   |   |   |   |   |   |   |

STEP 0 / 7

<

>

>>

SPEED: 2X 4X 8X 32X

RESULTS

CONFIGURATION  
Algorithm : GBFS + H2 (Euclidean)

METRICS  
 Status : DITEMUKAN  
 Waktu : 0.19 ms  
 Iterasi : 8  
 Total Cost: 112  
 Langkah : 7

URUTAN GERAKAN  
URULDL

DETAIL LANGKAH  
 Step : 1 / 7  
 arah : ATAS (U)  
 dari : (5, 4)  
 ke : (2, 4)  
 cost : 24  
 Total Cost: 24

#### ❖ Kasus Uji 4

|                   |
|-------------------|
| Input             |
| 10 10<br>*****X** |

```

*X**X**
***X*****
*****X*
**X*****
*****X**
****X*****
*Z*****X*
*****
*****OX**

```

```

1 2 3 4 5 6 7 999 8 9
1 999 2 3 4 999 5 6 7 8
1 2 3 4 999 5 6 7 8 9
1 2 3 4 5 6 7 8 999 0
1 2 999 3 4 5 6 7 8 9
1 2 3 4 5 6 7 999 8 9
1 2 3 4 999 5 6 7 8 9
1 2 3 4 5 6 7 8 999 9
1 2 3 4 5 6 7 8 9 0
1 2 3 4 5 6 7 999 9 0

```

## Output UCS

Slider Puzzle Solver

### ICE SLIDING PUZZLE SOLVER

INPUT FILE

test4.txt

ALGORITHM

☒ UCS
☐ GBFS
☐ A\*

HEURISTIC

☐ H1 - Manhattan
☐ H2 - Euclidean
☐ H3 - MinSlidecost

RUN

SAVE RESULT

solusi tidak ditemukan.

|   | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|---|---|---|---|---|---|---|---|---|---|---|
| 0 |   |   |   |   |   |   |   |   |   |   |
| 1 |   |   |   |   |   |   |   |   |   |   |
| 2 |   |   |   |   |   |   |   |   |   |   |
| 3 |   |   |   |   |   |   |   |   |   |   |
| 4 |   |   |   |   |   |   |   |   |   |   |
| 5 |   |   |   |   |   |   |   |   |   |   |
| 6 |   |   |   |   |   |   |   |   |   |   |
| 7 |   |   |   |   |   |   |   |   |   |   |
| 8 |   |   |   |   |   |   |   |   |   |   |
| 9 |   |   |   |   |   |   |   |   |   |   |

step 0 / 0

speed: 2% 4% 8% 32%

### RESULTS

CONFIGURATION

Algorithm : UCS

METRICS

Status : TIDAK DITEMUKAN

Waktu : 0.74 ms

Iterasi : 18

Tidak ada Jalur yang valid menuju

## Output GBFS

Slider Puzzle Solver

ICE SLIDING PUZZLE SOLVER

INPUT FILE

test4.txt

ALGORITHM

☐ UCS
☒ GBFS
☐ A\*

HEURISTIC

☐ H1 - Manhattan
☒ H2 - Euclidean
☐ H3 - Minslidelcost

RUN

SAVE RESULT

solusi tidak ditemukan.

|   | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |   | 8 | 9 |
| 1 |   |   |   |   |   |   |   |   |   |   |
| 2 |   |   |   |   |   |   |   |   |   |   |
| 3 |   |   |   |   |   |   |   |   |   |   |
| 4 |   |   |   |   |   |   |   |   |   |   |
| 5 |   |   |   |   |   |   |   |   |   |   |
| 6 |   |   |   |   |   |   |   |   |   |   |
| 7 |   |   |   |   |   |   |   |   |   |   |
| 8 |   |   |   |   |   |   |   |   |   |   |
| 9 |   |   |   |   |   |   |   |   |   |   |

STEP 0 / 0

<

<

>

>

SPEED: 2X 4X 8X 32X

RESULTS

CONFIGURATION

Algorithm : GBFS + H2 (Euclidean)

METRICS

Status : TIDAK DITEMUKAN

Waktu : 0.83 ms

Iterasi : 18

Tidak ada jalur yang valid menuju

Output A\*

Slider Puzzle Solver

ICE SLIDING PUZZLE SOLVER

INPUT FILE

test4.txt

ALGORITHM

☐ UCS
☐ GBFS
☒ A\*

HEURISTIC

☐ H1 - Manhattan
☐ H2 - Euclidean
☒ H3 - Minslidelcost

RUN

SAVE RESULT

solusi tidak ditemukan.

|   | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |   | 8 | 9 |
| 1 |   |   |   |   |   |   |   |   |   |   |
| 2 |   |   |   |   |   |   |   |   |   |   |
| 3 |   |   |   |   |   |   |   |   |   |   |
| 4 |   |   |   |   |   |   |   |   |   |   |
| 5 |   |   |   |   |   |   |   |   |   |   |
| 6 |   |   |   |   |   |   |   |   |   |   |
| 7 |   |   |   |   |   |   |   |   |   |   |
| 8 |   |   |   |   |   |   |   |   |   |   |
| 9 |   |   |   |   |   |   |   |   |   |   |

STEP 0 / 0

<

<

>

>

SPEED: 2X 4X 8X 32X

RESULTS

CONFIGURATION

Algorithm : A\* + H3 (Minslidelcost)

METRICS

Status : TIDAK DITEMUKAN

Waktu : 0.48 ms

Iterasi : 18

Tidak ada jalur yang valid menuju

## ❖ Kasus Uji 5

| Input                             |
|-----------------------------------|
| 10 10<br>**X*X**X**<br>*XO**X**** |

```

***X*****
*****X*
*1X*****
X***0**X**
****X*****
*Z*****X*
*****
*****X**

```

```

1 2 999 4 5 6 7 999 8 9
1 999 2 999 4 999 5 6 7 8
1 2 3 4 999 5 6 7 8 9
1 2 3 4 5 6 7 8 999 0
1 2 999 3 4 5 6 7 8 9
1 999 3 4 5 6 7 999 8 9
1 2 3 4 999 5 6 7 8 9
1 2 3 4 5 6 7 8 999 9
1 2 3 4 5 6 7 8 9 0
1 2 3 4 5 6 7 999 9 0

```

## Output UCS

**ICE SLIDING PUZZLE SOLVER**

**INPUT FILE**

tests.txt

**ALGORITHM**

☒ UCS

☐ GBFS

☐ A\*

**HEURISTIC**

☐ H1 - Manhattan

☐ H2 - Euclidean

☐ H3 - Minslidecost

**RUN**

**SAVE RESULT**

**SOLUSI DITEMUKAN!**

|   | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|---|---|---|---|---|---|---|---|---|---|---|
| 0 |   |   |   |   |   |   |   |   |   |   |
| 1 |   |   | 0 |   |   |   |   |   |   |   |
| 2 |   |   |   |   |   |   |   |   |   |   |
| 3 |   |   |   |   |   |   |   |   |   |   |
| 4 |   | 1 |   |   |   |   |   |   |   |   |
| 5 |   |   |   |   | 0 |   |   |   |   |   |
| 6 |   |   |   |   |   |   |   |   |   |   |
| 7 |   |   |   |   |   |   |   |   |   |   |
| 8 |   |   |   |   |   |   |   |   |   |   |
| 9 |   |   |   |   |   |   |   |   |   |   |

STEP 0 / 10

< > >> <<

SPEED: 25% 50% 75% 100%

**RESULTS**

**CONFIGURATION**

Algorithm: UCS

**METRICS**

Status : DITEMUKAN

Waktu : 0.75 ms

Iterasi : 17

Total Cost: 2103

Langkah : 10

**URUTAN GERAKAN**

RULULDLURU

**DETAIL LANGKAH**

Step : 1 / 10

Arah : KANAN (R)

Dari : (7, 1)

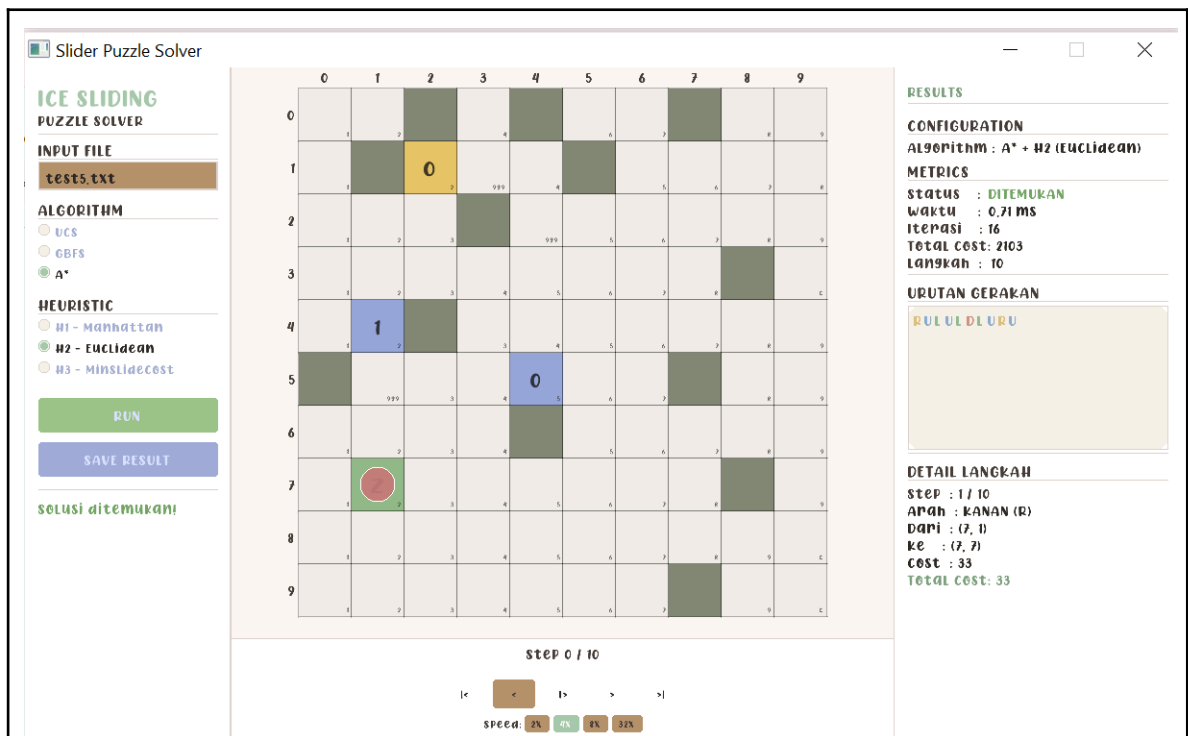
ke : (7, 7)

Cost : 33

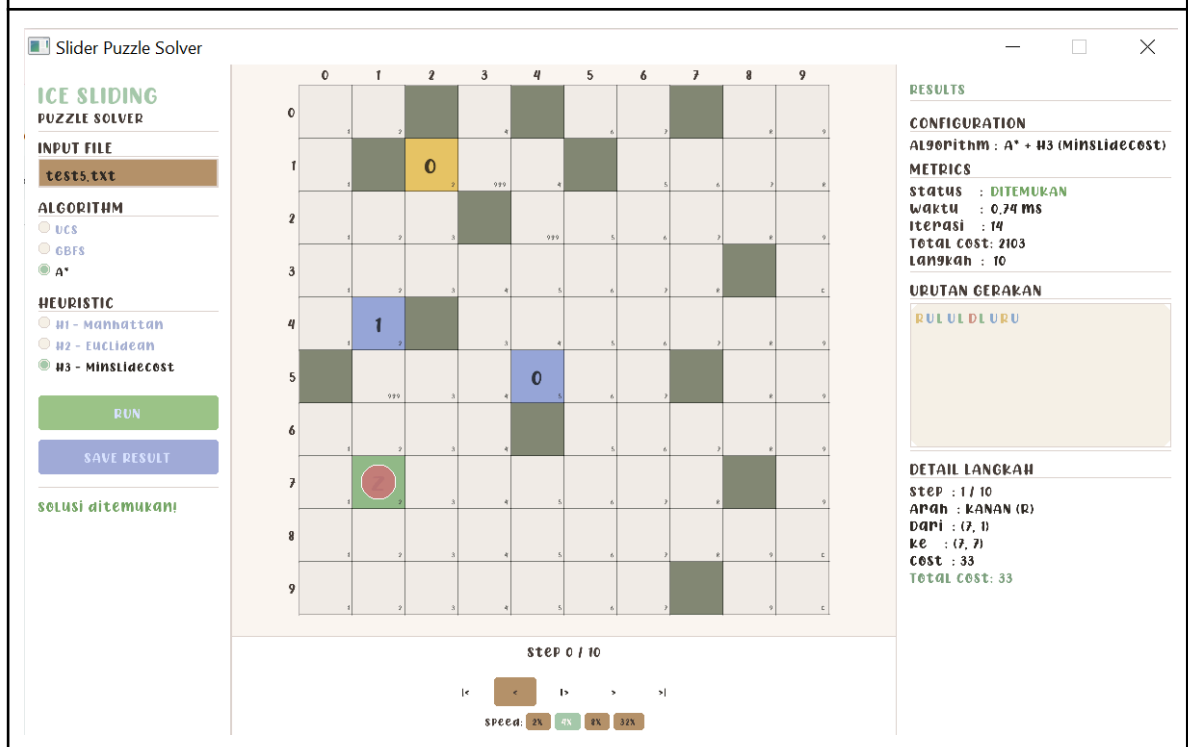
Total Cost: 33

## Output GBFS





Output A\*



## ❖ Kasus Uji 6

| Input                                 |
|---------------------------------------|
| <pre> 10 10 *****X** *X**X**** </pre> |

```

***X*****
*****X*
**X*****
*****X**
****X*****
*Z*****X*
*****OX*
*****XX**

1 2 3 4 5 6 7 999 8 9
1 999 2 3 4 999 0 0 7 8
1 0 0 999 2 5 0 0 8 9
1 0 0 0 0 0 0 0 999 0
1 0 999 3 4 5 0 7 8 9
1 0 3 4 5 6 0 999 8 9
1 0 3 4 999 5 0 7 8 9
1 2 0 0 0 1 0 0 999 9
1 2 3 4 5 6 0 0 999 0
1 2 3 4 5 6 999 999 9 0

```

## Output UCS

**Slider Puzzle Solver**

**ICE SLIDING PUZZLE SOLVER**

**INPUT FILE**  
test6.txt

**ALGORITHM**  
☒ UCS  
☐ GBFS  
☐ A\*

**HEURISTIC**  
☐ H1 - Manhattan  
☐ H2 - Euclidean  
☐ H3 - Minslidecost

**RUN**

**SAVE RESULT**

**SOLUSI DITEMUKAN!**

|   | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |   | 8 | 9 |
| 1 |   |   |   |   |   |   |   |   |   |   |
| 2 |   |   |   |   |   |   |   |   |   |   |
| 3 |   |   |   |   |   |   |   |   |   |   |
| 4 |   |   |   |   |   |   |   |   |   |   |
| 5 |   |   |   |   |   |   |   |   |   |   |
| 6 |   |   |   |   |   |   |   |   |   |   |
| 7 |   |   |   |   |   |   |   |   |   |   |
| 8 |   |   |   |   |   |   |   |   |   |   |
| 9 |   |   |   |   |   |   |   |   |   |   |

Step 0 / 8

Speed: 2X 4X 8X 32X

**RESULTS**

**CONFIGURATION**  
Algorithm: UCS

**METRICS**  
 Status : DITEMUKAN  
 Waktu : 0.58 ms  
 Iterasi : 9  
 Total Cost: 0  
 Langkah : 8

**URUTAN GERAKAN**  
URDRULDR

**DETAIL LANGKAH**  
 Step : 1 / 8  
 Arah : ATAS (U)  
 Dari : (7, 0)  
 Ke : (2, 0)  
 Cost : 0  
 Total Cost: 0

## Output GBFS

Slider Puzzle Solver

ICE SLIDING PUZZLE SOLVER

INPUT FILE

test6.txt

ALGORITHM

☐ UCS
☒ GBFS
☐ A\*

HEURISTIC

☒ H1 - Manhattan
☐ H2 - Euclidean
☐ H3 - Minslidedcost

RUN

SAVE RESULT

SOLUSI DITEMUKAN!

|   | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|---|---|---|---|---|---|---|---|---|---|---|
| 0 |   |   |   |   |   |   |   |   |   |   |
| 1 |   |   |   |   |   |   |   |   |   |   |
| 2 |   |   |   |   |   |   |   |   |   |   |
| 3 |   |   |   |   |   |   |   |   |   |   |
| 4 |   |   |   |   |   |   |   |   |   |   |
| 5 |   |   |   |   |   |   |   |   |   |   |
| 6 |   |   |   |   |   |   |   |   |   |   |
| 7 |   |   |   |   |   |   |   |   |   |   |
| 8 |   |   |   |   |   |   |   |   |   |   |
| 9 |   |   |   |   |   |   |   |   |   |   |

STEP 0 / 2

<

>

>>

SPEED: 2X 4X 8X 32X

RESULTS

CONFIGURATION

Algorithm: GBFS + H1 (Manhattan)

METRICS

Status : DITEMUKAN  
Waktu : 0.54 ms  
Iterasi : 3  
Total Cost: 1  
Langkah : 2

URUTAN GERAKAN

RD

DETAIL LANGKAH

Step : 1 / 2  
Arah : KANAN (R)  
Dari : (7, 0)  
Ke : (7, 7)  
Cost : 1  
Total Cost: 1

## Output A\*

Slider Puzzle Solver

ICE SLIDING PUZZLE SOLVER

INPUT FILE

test6.txt

ALGORITHM

☐ UCS
☐ GBFS
☒ A\*

HEURISTIC

☐ H1 - Manhattan
☐ H2 - Euclidean
☒ H3 - Minslidedcost

RUN

SAVE RESULT

SOLUSI DITEMUKAN!

|   | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|---|---|---|---|---|---|---|---|---|---|---|
| 0 |   |   |   |   |   |   |   |   |   |   |
| 1 |   |   |   |   |   |   |   |   |   |   |
| 2 |   |   |   |   |   |   |   |   |   |   |
| 3 |   |   |   |   |   |   |   |   |   |   |
| 4 |   |   |   |   |   |   |   |   |   |   |
| 5 |   |   |   |   |   |   |   |   |   |   |
| 6 |   |   |   |   |   |   |   |   |   |   |
| 7 |   |   |   |   |   |   |   |   |   |   |
| 8 |   |   |   |   |   |   |   |   |   |   |
| 9 |   |   |   |   |   |   |   |   |   |   |

STEP 0 / 8

<

>

>>

SPEED: 2X 4X 8X 32X

RESULTS

CONFIGURATION

Algorithm: A\* + H3 (Minslidedcost)

METRICS

Status : DITEMUKAN  
Waktu : 0.44 ms  
Iterasi : 9  
Total Cost: 0  
Langkah : 8

URUTAN GERAKAN

URDRULDR

DETAIL LANGKAH

Step : 1 / 8  
Arah : ATAS (U)  
Dari : (7, 0)  
Ke : (2, 0)  
Cost : 0  
Total Cost: 0

## ❖ Kasus Uji 7

| Input                                       |
|---------------------------------------------|
| 14 13<br>XXXXXXXXXXXXXXXXX<br>XXX*****O**XX |

```

XXX*0*X*****X
X****X****1*X
X*XXXXX*****X
X7X*L*****X*X
X*X**X**2***X
X*X*XX*X*XX*X
X*****3***X*X
X*XX**X**4**X
X***X*****X*X
X*5***6**X8*X
X**XX*9**XZ*X
XXXXXXXXXXXXXXXXX
999 999 999 999 999 999 999 999 999 999 999 999
999 999 999 1 1 1 1 1 1 1 1 999 999
999 999 999 1 1 1 999 1 1 8 8 8 999
999 1 1 1 9 999 4 4 1 6 1 6 999
999 1 999 999 999 999 999 4 1 6 6 6 999
999 1 999 1 999 4 4 4 1 6 999 3 999
999 1 999 1 1 999 4 4 1 1 1 1 999
999 1 999 1 999 999 1 999 1 999 999 1 999
999 2 2 2 2 1 1 1 1 5 999 5 999
999 1 999 999 1 1 999 1 1 1 5 5 999
999 2 2 2 999 1 1 1 1 1 999 5 999
999 2 1 2 1 1 1 1 1 999 1 1 999
999 2 2 999 999 1 1 1 1 999 1 1 999
999 999 999 999 999 999 999 999 999 999 999 999

```

Output UCS

Slider Puzzle Solver

ICE SLIDING PUZZLE SOLVER

INPUT FILE

test14.txt

ALGORITHM

☒ UCS
☐ GBFS
☐ A\*

HEURISTIC

☐ H1 - Manhattan
☐ H2 - Euclidean
☐ H3 - Minslidecost

RUN

SAVE RESULT

SOLUSI DITEMUKAN!

0 1 2 3 4 5 6 7 8 9 10 11 12

0

0

1

0

2

3

1

4

5

7

6

7

8

9

10

11

12

13

STEP 0 / 48

<

>

SPEED: 2X 4X 8X 32X

RESULTS

CONFIGURATION

Algorithm: UCS

METRICS

Status : DITEMUKAN

Waktu : 1.08 ms

Iterasi : 225

Total Cost: 372

Langkah : 48

URUTAN GERAKAN

R U L U L D R U R D L D R L D R D L U L D R U

R D R L U R U R D R D L U R U L D L D R D L D

R U

DETAIL LANGKAH

Step : 1 / 48

Arah : KANAN (R)

Dari : (12, 10)

ke : (12, 11)

Cost : 1

Total Cost: 1

## Output GBFS

Slider Puzzle Solver

ICE SLIDING PUZZLE SOLVER

INPUT FILE

test14.txt

ALGORITHM

☐ UCS
☒ GBFS
☐ A\*

HEURISTIC

☒ H1 - Manhattan
☐ H2 - Euclidean
☐ H3 - Minslidecost

RUN

SAVE RESULT

SOLUSI DITEMUKAN!

0 1 2 3 4 5 6 7 8 9 10 11 12

0

0

1

0

2

3

1

4

5

7

6

7

8

9

10

11

12

13

STEP 0 / 57

<

>

SPEED: 2X 4X 8X 32X

RESULTS

CONFIGURATION

Algorithm: GBFS + H1 (Manhattan)

METRICS

Status : DITEMUKAN

Waktu : 0.93 ms

Iterasi : 63

Total Cost: 408

Langkah : 57

URUTAN GERAKAN

R U L U L D R U R D L D L U R D L D L U L

D R U R D L U D R U R D R U R D L U D R

U L D L D R D L D R U

DETAIL LANGKAH

Step : 1 / 57

Arah : KANAN (R)

Dari : (12, 10)

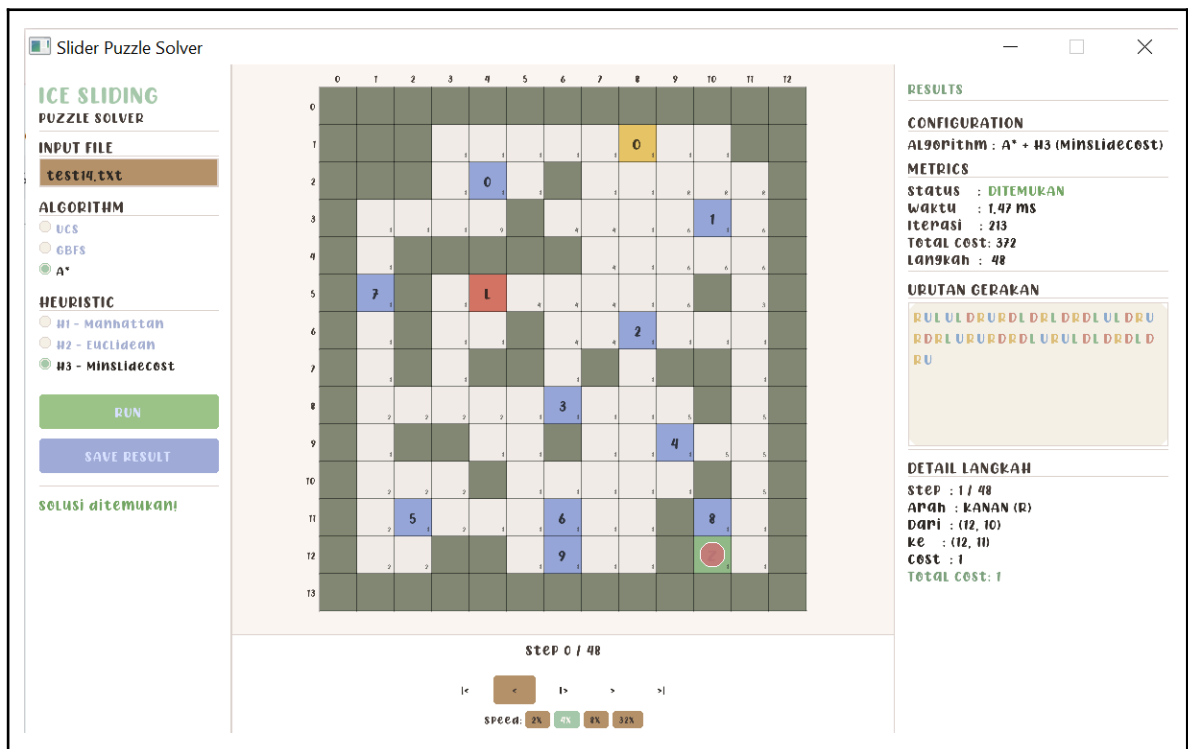
ke : (12, 11)

Cost : 1

Total Cost: 1

## Output A\*

34



## ❖ Kasus Uji 8

### Input

25 25

```

XXXXXXXXXXXXXXXXXXXXXXXXXXXXX
X**X***XX***X*****XX
X*X*X***X***X*X*****X*X
X***XX*****XX***X
X*X**0**X*****Z***X*X
X*X*****X***2**X**X***X
X*****X***O**X***X
X*****XX***X*****XX
X**X*****X
XX*****X
X*X*****X***X*****X
XX*X*****X***X**X
X**3*****XX**X
X**X*****XX*****X**X
X**X*****X*L*****X*LX
X*****X*****L**X
X***X*****XX*****X
X*X***X***X*****X***X
X*****X*****1*****X
X*****X**X*****X***X
X*****X*****XX***XX***XX
X*****X*****X
X**X*****X*****X
X*****X*****X**X

```



Slider Puzzle Solver

ICE SLIDING PUZZLE SOLVER

INPUT FILE

test25.txt

ALGORITHM

☒ UCS
☐ GBFS
☐ A\*

HEURISTIC

☐ H1 - Manhattan
☐ H2 - Euclidean
☐ H3 - Minslidecost

RUN

SAVE RESULT

SOLUSI DITEMUKAN!

RESULTS

CONFIGURATION

Algorithm: UCS

METRICS

Status : DITEMUKAN  
Waktu : 4.22 ms  
Iterasi : 414  
Total Cost: 872  
Langkah : 32

URUTAN GERAKAN

RDRULURIDLRURDRURULULU  
RURDLRULU

DETAIL LANGKAH

Step : 1 / 32  
Arah : KANAN (R)  
Dari : (4, 18)  
Ke : (4, 20)  
Cost : 22  
Total Cost: 22

STEP 0 / 32

<

>

>>

SPEED: 2X 4X 8X 32X

## Output GBFS

Slider Puzzle Solver

ICE SLIDING PUZZLE SOLVER

INPUT FILE

test25.txt

ALGORITHM

☐ UCS
☒ GBFS
☐ A\*

HEURISTIC

☐ H1 - Manhattan
☒ H2 - Euclidean
☐ H3 - Minslidecost

RUN

SAVE RESULT

SOLUSI DITEMUKAN!

RESULTS

CONFIGURATION

Algorithm: GBFS + H2 (Euclidean)

METRICS

Status : DITEMUKAN  
Waktu : 4.73 ms  
Iterasi : 107  
Total Cost: 1232  
Langkah : 47

URUTAN GERAKAN

LURDLRULURDRURULDLURDL  
DLULULDRURULULURDLRUL  
U

DETAIL LANGKAH

Step : 1 / 47  
Arah : KIRI (L)  
Dari : (4, 18)  
Ke : (4, 9)  
Cost : 54  
Total Cost: 54

STEP 0 / 47

<

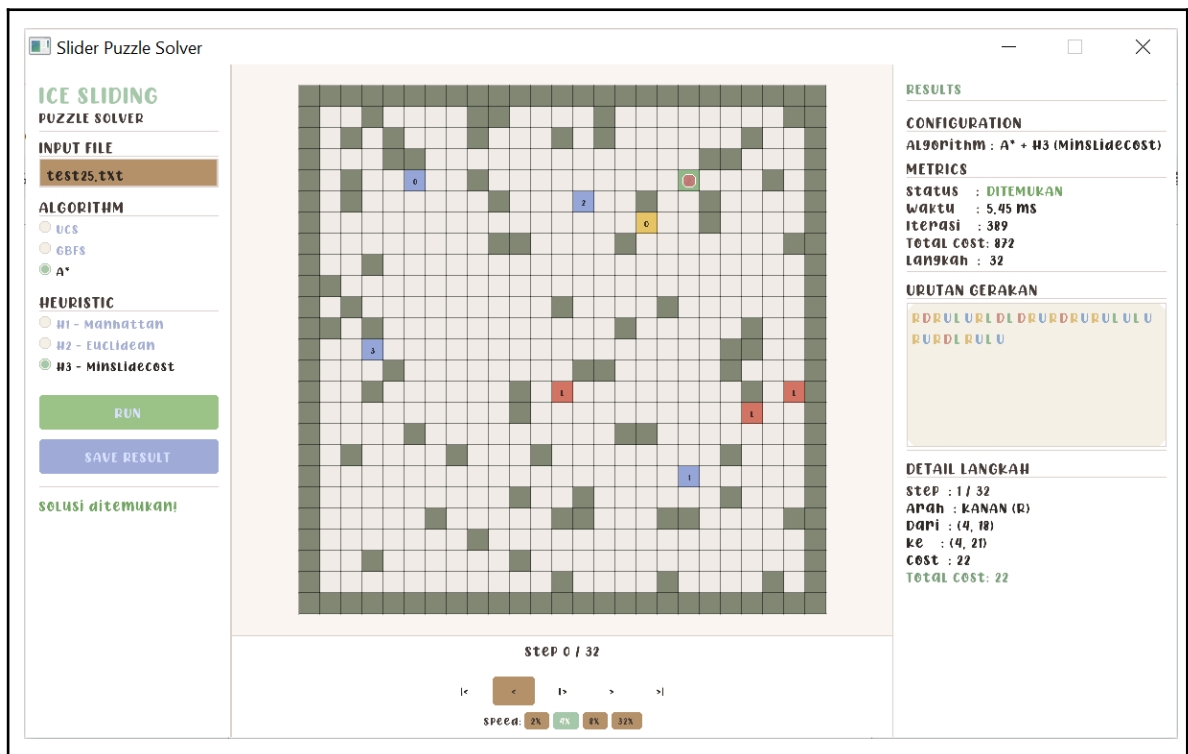
>

>>

SPEED: 2X 4X 8X 32X

## Output A\*





## ❖ Kasus Uji 9

### Input

### 50 50

```

XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
X*XXX*****X**X*****X**X*****X
X*****2*****X*X*****X*****X*****XXX*X*X
XXX*****X*****L*****X**X**XX*****X
XX*****X*****X**XX**X**XX*****X*X**X*XX
X*****X**XX**XX**X**X**X*****XX*****X
X*****X**XX*****X*****X**X**X*****X
X**X*****X**X**X*****X**X*****X*****X
X**XX**X*****X**XX*****XX*****X**X
X*****X**XXX*****X*****XX*****XX
X*****X**XX**X**X*****X*****XX**X*****XX
X*X*****X*3**X*****X**X**X*****X**X*****X**XX
X*****X*****X*****X**X**XX
X*****X*****X*****X*****X**X**X*****X
X**X*****X*****X**X*****X*****X**X*****X
X*****X*****XX*****X**X**X*****X**X*****X
XXX*X*****X**X*****XX*****X*****X**X
X*****X*****X*****X*L*X**XX**X**X
X**X**X**X*****X*****X**X*****XX*****X**X**X
X*X*****X*****X*****X*****X**X**X*****X*****X
X*****X*****X*****X*****X*****X*****X
XX*****X**XX*XXX*****X**XX**XX*****X**X
X*****XO*X*****XX**X**X*****X*****XXX

```

XX\*\*\*\*\*LX\*X\*X\*\*X\*X\*\*XX\*X\*X\*\*X\*X\*\*\*X\*\*\*X\*\*\*\*\*X  
X\*\*\*\*\*X\*\*\*\*\*X\*\*\*\*\*X\*\*X\*\*\*\*\*X\*X\*\*\*\*\*X\*\*\*\*\*XX  
X\*\*\*\*\*X\*\*\*\*\*X\*\*\*\*\*X\*\*X\*\*X\*\*\*\*\*X\*\*\*\*\*X  
X\*\*\*X\*X\*X\*\*\*\*\*1\*\*\*\*X\*\*\*\*\*X\*\*\*\*\*X\*\*\*\*\*XXX\*\*\*\*\*XX  
X\*XX\*\*\*\*\*X\*\*\*X\*\*\*\*\*X\*XXX\*X\*\*\*\*\*X\*X\*\*\*X  
X\*\*\*\*X\*\*\*\*\*X\*\*\*\*\*X\*X\*\*\*\*\*X\*\*\*\*\*X\*X\*\*\*\*\*X  
X\*\*\*\*\*X\*X\*\*\*X\*\*\*\*\*X\*\*\*X\*\*\*\*\*X\*\*\*\*\*X\*\*\*\*\*X\*X  
XX\*\*\*\*\*X\*\*X\*\*X\*\*X\*\*\*\*\*X\*\*\*\*\*X\*\*\*\*\*X\*\*\*\*\*X  
X\*XX\*\*\*XXX\*\*\*\*\*XX\*X\*\*X\*\*\*\*\*X\*XX\*XX\*\*\*\*\*X\*\*\*X\*X  
X\*\*\*\*\*XL\*\*\*\*\*X\*\*\*\*\*X\*\*\*XX\*\*X  
X\*\*\*\*\*X\*\*X\*\*\*\*\*X\*X\*X\*\*\*\*\*X\*\*\*\*\*X\*\*X\*X\*\*\*\*\*XX  
X\*\*\*\*\*X\*\*\*\*\*X\*X\*\*\*\*\*X\*\*\*X\*\*\*\*\*X\*\*X  
X\*X\*\*\*\*\*X\*\*\*\*\*X\*X\*\*\*\*\*X\*\*\*\*\*X\*\*\*\*\*ZX  
X\*\*X\*\*\*X\*\*\*\*\*X\*\*XX\*\*\*\*\*XX\*\*X\*\*\*X\*\*\*\*\*X  
X\*X\*XX\*\*\*X\*\*\*\*\*XX\*\*\*\*\*X\*\*\*\*\*X\*\*\*X\*X\*\*\*X\*\*\*X  
X\*\*\*\*X\*\*\*X\*X\*\*\*\*\*X\*\*\*\*\*X\*\*\*\*\*X0\*\*\*\*\*X\*\*\*XXX  
X\*\*\*\*\*X\*\*\*\*\*X\*\*\*\*\*XX\*\*\*\*\*X\*\*\*\*\*X\*X  
X\*X\*\*X\*X\*X\*\*\*\*\*X\*X\*X\*\*\*\*\*X\*\*\*\*\*X  
X\*\*\*XX\*\*X\*X\*\*\*\*\*X\*\*\*\*\*X\*\*\*X\*\*\*\*\*XX\*\*X  
X\*\*\*\*X\*\*\*\*\*X\*\*\*\*\*X\*\*X\*\*\*\*\*X\*\*\*X\*\*\*X\*\*\*XX\*\*\*X  
X\*\*X\*\*\*\*\*X\*X\*\*\*\*\*X\*\*\*\*\*X\*\*\*\*\*X\*\*\*\*\*X\*\*\*X\*XX  
X\*\*X\*\*\*X\*\*\*\*\*X\*\*X\*X\*\*\*\*\*XX\*\*X\*\*\*\*\*X\*\*\*\*\*X  
X\*\*X\*\*X\*\*\*\*\*X\*\*\*\*\*XX\*\*\*\*\*X\*\*X\*\*\*\*\*XX  
XX\*\*XX\*\*\*X\*\*\*\*\*XXX\*\*\*\*\*X\*\*\*\*\*X  
X\*\*\*XX\*\*\*\*\*X\*X\*\*X\*\*\*X\*\*\*L\*\*\*X\*\*\*X  
X\*\*\*\*\*X\*\*X\*\*X\*\*\*\*\*XX\*\*XX\*XX\*X\*\*\*X

XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX  
999 999 999 999 999 999 999 999 999 999 999 999 999 999 999  
999 999 999 999 999 999 999 999 999 999 999 999 999 999 999  
999 999 999 999 999 999 999 999 999 999 999 999 999 999  
999 9 999 999 999 5 3 7 2 2 6 7 4 4 7 999 1 3 2 999 3 6 6 7 5 1 8 7 1 9 1 1 2 1  
2 9 2 999 9 4 999 1 9 3 5 9 8 1 999  
999 2 9 9 5 8 3 4 4 6 2 8 2 999 1 999 8 8 1 1 8 8 4 2 7 6 999 7 6 3 5 7 8 4 5 8 6  
999 7 9 5 7 6 999 999 999 8 999 1 999  
999 999 999 3 4 6 4 2 5 2 8 6 2 6 1 7 4 999 9 5 9 4 3 999 1 1 5 3 8 9 5 2 999 8 7  
4 999 7 2 6 999 999 7 8 3 5 5 1 5 999  
999 999 4 1 5 6 1 8 7 999 5 7 3 2 1 999 7 9 9 6 999 999 2 999 999 9 4 999 8 9 9  
9 999 999 3 4 6 9 8 7 6 999 7 999 5 7 999 3 999 999  
999 3 4 8 6 9 5 7 6 7 5 3 7 1 999 1 7 999 999 9 8 6 999 999 1 999 5 999 7 999 7  
4 7 999 4 6 1 2 5 2 999 999 9 2 7 7 5 6 7 999  
999 6 1 6 7 3 9 999 3 7 999 999 1 5 5 6 1 999 999 8 4 999 2 5 9 2 7 8 9 9 7 6  
999 9 4 8 999 4 999 7 8 999 6 4 7 5 9 7 7 999  
999 2 6 999 1 1 4 5 1 2 8 999 6 8 4 999 7 1 999 5 8 7 6 3 6 6 6 4 6 5 7 999 1  
999 4 5 9 1 3 7 2 5 999 8 2 1 1 3 999  
999 5 3 999 999 8 9 7 9 999 2 4 8 6 7 9 4 4 5 5 2 3 8 999 2 4 9 999 999 7 8 9 7 4  
5 5 1 7 7 999 999 5 9 9 3 1 999 7 6 999  
999 2 3 5 7 3 3 3 8 4 4 9 7 7 999 5 2 5 999 999 999 5 8 8 1 1 4 1 7 1 999 2 9 7 3  
8 999 999 5 8 6 7 8 8 2 8 9 4 999 999  
999 5 1 1 7 1 9 1 9 2 3 999 1 8 999 999 4 6 999 8 999 6 8 6 2 7 8 9 999 5 2 6 1 8

9 999 999 9 999 6 9 8 2 4 8 2 4 4 999 999  
 999 5 999 3 8 8 4 8 6 5 999 9 3 8 6 999 4 1 9 1 5 3 999 8 3 999 8 5 999 9 3 9 8 6  
 8 999 4 2 999 4 7 9 2 4 999 6 1 1 999 999  
 999 9 9 6 2 1 1 8 6 9 8 6 3 2 4 3 1 2 4 5 3 3 4 2 6 6 7 7 7 4 4 999 7 7 5 3 7 3 8  
 4 3 999 8 1 999 3 1 999 999  
 999 1 1 4 6 3 6 7 6 4 2 7 999 6 3 2 6 7 2 9 4 2 5 9 5 9 3 6 999 5 5 7 1 5 1 5 999 7  
 3 5 2 999 6 7 999 8 8 9 6 999  
 999 3 6 2 999 2 3 6 5 7 2 2 1 999 8 9 8 6 9 3 999 3 7 6 999 3 7 9 4 4 999 1 8 4 9  
 2 9 6 2 1 2 4 9 2 8 7 9 999 3 999  
 999 6 5 1 9 999 6 7 8 3 1 999 999 2 5 5 9 9 2 1 999 7 2 2 999 1 999 9 2 5 2 8 8 6  
 4 8 6 4 3 999 6 1 999 2 3 1 999 2 5 999  
 999 999 999 8 999 4 8 5 9 7 3 5 999 6 7 5 999 2 8 6 3 6 1 999 999 6 9 4 1 1 1 4  
 9 5 999 5 2 1 4 9 8 4 4 7 7 6 6 999 9 999  
 999 5 8 5 5 5 3 5 999 4 9 9 1 5 3 9 9 6 1 8 2 1 2 5 4 4 4 2 2 6 3 9 9 5 3 999 5 999  
 7 999 2 999 999 7 7 5 999 2 2 999  
 999 1 4 999 9 999 3 9 999 8 6 9 6 999 6 7 1 5 1 6 1 1 2 7 999 5 8 7 999 7 7 5 9  
 999 999 5 3 5 4 4 7 999 6 8 8 999 8 999 8 999  
 999 6 999 4 6 1 6 1 999 9 5 2 2 7 9 999 8 8 9 9 5 7 999 8 6 9 9 2 6 7 999 5 999 2  
 9 9 999 8 3 8 3 999 9 6 1 5 8 8 3 999  
 999 3 8 1 4 9 7 5 999 5 1 8 1 5 2 9 3 9 999 1 6 8 8 4 8 1 7 7 1 5 4 999 6 6 3 8 9 7  
 5 6 6 9 1 8 8 5 2 8 3 999  
 999 999 4 5 1 9 9 6 6 5 3 999 2 4 999 999 6 3 999 999 999 7 3 7 7 9 8 4 5 2 3 6  
 999 2 8 999 999 5 3 999 999 5 5 2 7 999 6 3 2 999  
 999 4 3 3 4 1 9 1 6 1 3 2 5 1 999 1 6 999 1 5 5 1 3 8 4 4 6 7 999 999 5 2 999 2  
 999 2 8 1 3 5 999 3 2 9 6 2 8 999 999 999  
 999 999 8 4 8 5 9 999 999 6 999 5 999 1 1 7 999 6 4 999 3 3 999 999 2 999 7  
 999 8 3 999 2 2 999 2 9 8 6 999 1 9 5 999 9 9 9 3 8 1 999  
 999 8 2 4 5 2 6 6 5 8 9 2 8 4 1 999 1 8 7 2 1 999 3 7 999 9 5 1 5 5 999 1 999 5 2  
 2 9 5 8 3 9 3 999 3 8 2 9 1 999 999  
 999 9 6 7 1 2 9 7 5 8 6 2 5 6 3 2 6 5 3 8 1 999 7 7 999 7 8 999 9 7 8 4 9 7 1 999  
 7 4 1 7 8 5 5 6 3 2 5 2 5 999  
 999 7 1 8 2 999 9 999 7 999 1 1 3 4 2 3 9 7 2 6 1 4 999 4 8 3 8 999 3 5 1 1 999 3  
 9 3 3 9 4 3 999 999 999 9 8 3 7 4 999 999  
 999 9 999 999 4 9 3 9 4 5 3 3 4 999 9 6 8 999 1 2 6 5 1 2 8 6 4 8 9 999 4 999  
 999 999 4 999 9 3 9 5 9 2 6 999 4 999 5 5 2 999  
 999 1 6 7 1 999 3 7 3 2 8 999 6 2 9 5 999 4 999 6 1 3 6 9 999 9 7 3 1 4 1 8 3 8  
 999 2 999 1 1 2 8 5 2 7 9 4 1 7 2 999  
 999 7 3 9 6 8 3 8 7 4 3 7 9 999 4 999 7 7 4 999 7 3 1 8 999 4 5 9 999 5 3 7 2 5 2  
 999 4 5 6 4 6 4 999 4 1 9 4 999 3 999  
 999 999 5 5 4 1 6 8 5 999 1 6 999 8 9 999 9 4 999 5 2 8 9 3 2 8 999 1 4 9 6 9 1  
 999 6 7 8 2 7 5 2 999 6 3 7 2 9 2 8 999  
 999 3 999 999 7 9 6 999 999 999 7 1 8 7 3 4 7 6 4 1 999 999 9 999 1 2 999 7 4 5  
 7 999 6 999 999 6 999 999 5 5 9 5 7 999 2 8 4 999 8 999  
 999 5 4 6 8 4 1 7 2 8 999 999 7 4 1 5 2 4 1 7 4 4 4 9 9 999 4 2 2 4 2 2 6 5 5 8 9 3  
 9 7 8 999 6 8 1 999 999 6 5 999  
 999 9 8 2 6 8 999 5 5 999 8 9 7 5 1 8 999 6 999 7 999 1 7 8 2 5 6 9 3 999 4 5 7 2  
 2 3 999 6 6 999 6 999 3 4 1 1 2 2 999 999  
 999 8 1 2 1 2 8 4 4 5 999 4 8 5 4 4 9 4 7 4 2 5 999 7 999 7 3 4 3 999 9 8 2 999 5  
 8 999 9 1 2 9 2 4 9 7 9 999 1 3 999

999 5 999 1 9 6 7 3 3 999 6 2 5 8 1 2 9 7 1 4 7 4 5 7 8 2 1 5 999 7 999 1 6 3 4 2  
     5 7 8 7 6 4 8 5 4 2 6 5 1 999  
 999 9 5 7 999 9 6 3 999 2 8 9 8 9 999 5 6 999 999 5 4 9 5 4 7 6 7 999 999 7 4  
     999 4 1 4 999 5 5 4 3 6 5 9 9 6 8 3 2 1 999  
 999 5 999 9 999 999 5 2 9 999 4 5 2 2 5 8 7 999 999 4 3 4 3 2 999 1 4 6 4 3 999  
     6 2 3 7 999 6 5 999 9 999 8 8 7 8 999 8 9 5 999  
 999 3 4 3 7 999 9 3 3 999 5 2 999 3 2 9 2 1 999 8 8 6 7 3 8 1 8 6 7 999 9 1 9 7 7  
     7 999 1 4 5 2 7 2 999 7 4 2 999 999 999  
 999 1 4 5 8 9 4 2 3 8 5 4 7 2 1 4 6 4 1 6 3 999 5 8 1 4 4 999 7 3 4 6 8 1 999 999  
     9 3 7 3 5 999 8 4 3 4 7 999 8 999  
 999 9 999 3 3 999 7 3 999 4 999 5 4 6 4 999 5 7 999 4 999 4 8 999 7 3 2 3 7 7 8  
     7 3 999 8 4 9 7 7 6 8 3 3 9 4 3 4 1 7 999  
 999 2 4 8 999 999 9 6 999 2 999 7 3 999 2 7 3 9 6 7 6 7 9 999 7 3 8 2 9 4 3 8  
     999 6 2 5 999 6 6 8 5 4 2 3 2 999 999 1 8 999  
 999 1 5 3 7 999 9 3 9 6 999 6 6 3 6 999 6 1 7 999 6 999 999 6 6 6 6 3 999 5 9 3  
     999 9 9 7 999 6 5 1 999 6 3 2 999 999 1 3 5 999  
 999 6 2 999 1 7 9 2 6 2 3 999 8 999 1 8 4 7 9 5 7 1 999 9 2 9 3 4 5 3 999 2 9 5 3  
     9 999 6 9 4 7 4 999 4 3 8 999 3 999 999  
 999 5 1 999 8 3 8 999 8 5 9 1 6 6 999 7 3 999 6 999 3 7 4 5 8 7 3 9 3 3 8 999  
     999 7 5 999 3 3 8 6 4 9 999 2 5 1 6 4 3 999  
 999 1 7 999 7 4 999 1 4 3 4 1 1 8 9 2 999 6 7 4 2 1 3 2 5 3 999 999 5 7 7 6 999 7  
     3 9 9 999 6 1 999 6 1 6 8 5 6 8 999 999  
 999 999 4 6 999 999 9 3 7 999 3 7 8 8 5 1 2 1 9 6 999 999 999 1 5 4 2 8 8 9 3 6  
     4 6 5 3 2 1 5 7 8 999 2 1 5 1 1 9 5 999  
 999 1 1 2 999 999 9 5 6 6 3 5 1 4 7 1 5 5 1 3 9 5 3 3 7 4 8 999 6 999 1 6 999 1 4  
     5 1 999 4 8 6 999 6 6 3 999 1 3 4 999  
 999 9 3 2 7 3 999 7 9 999 9 5 999 5 7 9 2 5 9 9 5 6 2 3 7 9 2 9 9 3 3 5 5 4 999  
     999 6 9 999 999 6 999 999 6 999 5 7 6 7 999  
 999 999 999 999 999 999 999 999 999 999 999 999 999 999 999 999 999 999 999  
 999 999 999 999 999 999 999 999 999 999 999 999 999 999 999 999 999 999  
     999 999 999 999 999 999 999 999 999 999 999 999 999 999 999

Output UCS

Slider Puzzle Solver

ICE SLIDING  
PUZZLE SOLVER

INPUT FILE  
test50\_1.txt

ALGORITHM  
☒ UCS  
☐ GBFS  
☐ A\*

HEURISTIC  
☐ H1 - Manhattan  
☐ H2 - Euclidean  
☐ H3 - Minslidecost

RUN

SAVE RESULT

SOLUSI DITEMUKAN!

RESULTS

CONFIGURATION  
Algorithm : UCS

METRICS  
 Status : DITEMUKAN  
 Waktu : 22.36 ms  
 Iterasi : 1658  
 Total Cost: 1674  
 Langkah : 77

URUTAN GERAKAN

ULURULDLDLURURDLDLULULU  
 RDRULULULDLDLURULDRULU  
 RULULULURURDLDRURDL  
 LDLURURU

DETAIL LANGKAH

Step : 1 / 77  
 Arah : ATAS (U)  
 Dari : (35, 48)  
 Ke : (34, 48)  
 Cost : 3  
 Total Cost: 3

STEP 0 / 77

<

>

>>

SPEED: 2X 4X 8X 32X

## Output GBFS

Slider Puzzle Solver

ICE SLIDING  
PUZZLE SOLVER

INPUT FILE  
test50\_1.txt

ALGORITHM  
☐ UCS  
☒ GBFS  
☐ A\*

HEURISTIC  
☒ H1 - Manhattan  
☐ H2 - Euclidean  
☐ H3 - Minslidecost

RUN

SAVE RESULT

SOLUSI DITEMUKAN!

RESULTS

CONFIGURATION  
Algorithm : GBFS + H1 (Manhattan)

METRICS  
 Status : DITEMUKAN  
 Waktu : 16.92 ms  
 Iterasi : 206  
 Total Cost: 2007  
 Langkah : 79

URUTAN GERAKAN

LURDLDRDLULULURURURURU  
 RDRULULULDLDLURULDRULU  
 RULULULURULURDLDRURDL  
 LLDLURURU

DETAIL LANGKAH

Step : 1 / 79  
 Arah : KIRI (L)  
 Dari : (35, 48)  
 Ke : (35, 38)  
 Cost : 83  
 Total Cost: 83

STEP 0 / 79

<

>

>>

SPEED: 2X 4X 8X 32X

## Output A\*

Slider Puzzle Solver

ICE SLIDING  
PUZZLE SOLVER

INPUT FILE  
test50\_1.txt

ALGORITHM  
☐ UCS  
☐ GBFS  
☒ A\*

HEURISTIC  
☒ H1 - Manhattan  
☐ H2 - Euclidean  
☐ H3 - Minslidecost

RUN

SAVE RESULT

Solusi ditemukan!

RESULTS

CONFIGURATION  
Algorithm : A\* + H1 (Manhattan)

METRICS  
 Status : DITEMUKAN  
 Waktu : 28.46 ms  
 Iterasi : 1581  
 Total Cost: 1674  
 Langkah : 77

URUTAN GERAKAN  
 ULURULDLDLURURDLDLULULU  
 RDRULULDLDLULURDLRLU  
 RULULURULURDLDRURDL  
 LDLURURU

DETAIL LANGKAH  
 Step : 1 / 77  
 Arah : ATAS (U)  
 DAPI : (35, 48)  
 Ke : (34, 48)  
 Cost : 3  
 Total Cost: 3

STEP 0 / 77

<

>

>>

Speed: 2X 4X 8X 32X

## BAB 5

### KESIMPULAN

UCS dan A\* secara konsisten menghasilkan solusi dengan total cost yang sama, yang merupakan solusi optimal. Hal ini sesuai dengan teori bahwa UCS menjamin optimalitas pada graf berbobot positif, dan A\* menjamin optimalitas selama heuristik yang digunakan bersifat admissible.

Sebaliknya, GBFS tidak selalu menghasilkan solusi optimal. GBFS menemukan solusi dengan cost 408 lebih mahal 9,7% dari solusi optimal UCS (372) karena GBFS hanya meminimalkan  $h(n)$  tanpa mempertimbangkan biaya aktual  $g(n)$  yang sudah ditempuh. Ini konsisten dengan teori bahwa GBFS bersifat *greedy* dan tidak menjamin optimalitas.

## DAFTAR PUSTAKA

Rinaldi Munir. *Pengantar Strategi Algoritma*. Informatika, STEI ITB. Diakses 17 Februari 2026 dari <https://informatika.stei.itb.ac.id/~rinaldi.munir>.

GeeksForGeeks. *Uniform Cost Search (UCS) in AI*. Diakses 10 Mei 2026 dari <https://www.geeksforgeeks.org/artificial-intelligence/uniform-cost-search-ucs-in-ai/>

GeeksForGeeks. *Greedy Best-First Search in AI*. Diakses 10 Mei 2026 dari <https://www.geeksforgeeks.org/artificial-intelligence/greedy-best-first-search-in-ai/>



## LAMPIRAN

Tautan Repositori:

[https://github.com/SyaqinaOctavia/Tucil3\\_13524042\\_13524088](https://github.com/SyaqinaOctavia/Tucil3_13524042_13524088)

| No | Poin                                                                                             | Ya | Tidak |
|----|--------------------------------------------------------------------------------------------------|----|-------|
| 1  | Program berhasil dikompilasi tanpa kesalahan                                                     | ✓  |       |
| 2  | Program berhasil dijalankan                                                                      | ✓  |       |
| 3  | Solusi yang diberikan program benar dan mematuhi aturan permainan                                | ✓  |       |
| 4  | Program dapat membaca masukan berkas .txt serta menyimpan solusi dalam berkas .txt               | ✓  |       |
| 5  | Program memiliki Graphical User Interface (GUI)                                                  | ✓  |       |
| 6  | Ada algoritma pathfinding alternatif selain dari spesifikasi wajib (Algoritma X dan Algoritma Y) |    | ✓     |
| 7  | Ada tambahkan dua alternatif heuristik selain yang utama (Heuristik X dan Heuristik Y)           | ✓  |       |